

Дискретный анализ

Вайб-лекция 3: дерево, которое помнит, что лежит у него в поддереве

Андрей Калинин

Дискретный анализ, 2026/27

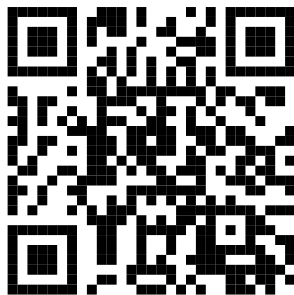
30 сентября 2026

Материалы лекций

Слайды всех лекций курса — в открытом репозитории:

`https://github.com/
alk-2000/da-lectures`

Сейчас там PDF первых двух лекций. Позже добавим исходники.



Конкурс по сортировке: регистрация открыта

Задача: самая быстрая сортировка 32-битных беззнаковых чисел. Точные условия и проверяющая программа — на следующей неделе.

Срок: 6 декабря 2026, 23:59 по Москве.

Призы:

- первое место — «отлично» на экзамене;
- ещё четыре места — +1 балл на экзамене.

После срока — семинар: победители рассказывают свои решения. Не можете объяснить своё решение — приза нет.



папка регистрации участников

Как записаться на конкурс

1. Откройте папку участников — QR-код на прошлом слайде, или в репозитории `alk-2000/da-lectures:2026/contest/sorting/participants`.
2. **Add file** → **Create new file**, имя — `<ваш ник>.md`; в файле одна строка — ваш GitHub-аккаунт, например `@octocat`.
3. **Propose new file**, затем **Create pull request**.
4. Напишите на **andrey@kalinin.ru** с темой «**Конкурс по сортировке**»: ФИО, группа, GitHub-аккаунт. В таблице результатов будут только ники.

Не выкладывайте решение в fork этого репозитория и не присылайте его pull request'ом до срока — это видят все. Для решения каждому дадим закрытый репозиторий.

План

Тема	Номер слайда
Удаление из AVL-дерева	8
2-3-4 деревья	12
Красно-чёрное дерево как 2-3-4	20
Удаление из красно-чёрного дерева	24
Красно-чёрное дерево с наклоном влево	32
Задача редактора: строка и колонка по смещению	37
Дерево отрезков	47
Дерево Фенвика	67
Декартово дерево (treap)	82
Неявное декартово дерево	93
Rope: текст в дереве	101
Как устроен буфер редактора Zed	105
Что ускорили внутри листа	109
Как мы знаем, что стало быстрее	114

Удаление: AVL

После удаления поддерево может стать ниже — и об этом надо сообщить родителю.

AVL-дерево: напоминание

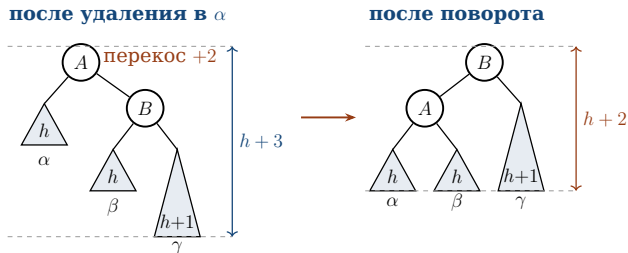
- **Определение.** Двоичное дерево поиска, в котором у каждого узла высоты левого и правого поддеревьев отличаются не больше чем на 1.
- **Что лежит в узле.** Высота его поддерева. Разность высот правого и левого поддеревьев назовём *перекосом*: у AVL-дерева он -1 , 0 или $+1$.
- **Почему это работает.** Высота AVL-дерева — $O(\log n)$, поэтому поиск, вставка и удаление — $O(\log n)$.
- **Как чинят.** После вставки или удаления высоты пересчитывают на пути к корню; где перекос стал ± 2 , делают поворот.

Удаление из AVL: подъём к корню

1. Удаляем, как в обычном дереве поиска. Если у узла два ребёнка, ставим на его место предшественника (наибольший ключ левого поддерева) и удаляем узел предшественника — у него не больше одного ребёнка.
2. Поднимаемся от места удаления к корню. В каждом узле пересчитываем перекос:
 - стал ± 1 (был 0) — высота поддерева не изменилась: **стоп**;
 - стал 0 (был ± 1) — поддерево стало ниже на 1: **идём выше**;
 - стал ± 2 — **поворот**, одинарный или двойной, как при вставке.

Время — $O(\log n)$: путь от листа до корня.

Поворот может не вернуть высоту



При вставке поворот всегда возвращает поддереву прежнюю высоту — подъём кончается. **При удалении** не всегда: было $h+3$, стало $h+2$, и родитель поддерева тоже должен пересчитать переко́с. Так может повториться на каждом уровне: до $O(\log n)$ поворотов за одно удаление. (Высота сохранится, только если β и γ обе высоты $h+1$.)

Как связано AVL с темой дня

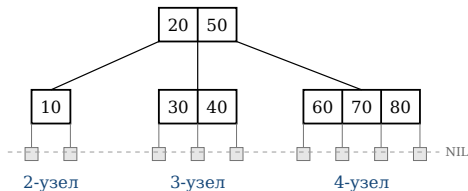
1. В узле лежит число про всё его поддерево — высота.
2. Удаление портит это число внизу, а чинят его снизу вверх: узел пересчитывает высоту и сообщает родителю, если она изменилась.
3. Весь день будут такие деревья: в узле — число про поддерево, после правки его чинят на пути к корню. Меняется только само число.

2-3-4 деревья

В узле от одного до трёх ключей. Удаление — заём у брата или слияние с ним.

2-3-4 дерево: определение

- Дерево поиска; в узле 1, 2 или 3 ключа. По числу детей это 2-, 3- и 4-узел.
- Детей на одного больше, чем ключей; ключи ребёнка лежат между соседними ключами родителя.
- **Все листья на одной глубине.** Поэтому высота — $O(\log n)$: каждый уровень хотя бы вдвое шире предыдущего.

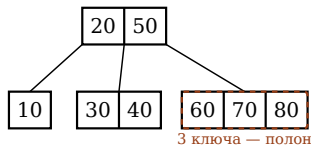


2-3-4 дерево — это B-дерево с $t = 2$

- В B-дереве с минимальной степенью t в каждом узле, кроме корня, от $t - 1$ до $2t - 1$ ключей; в корне — от 1.
- При $t = 2$ это от 1 до 3 ключей — ровно 2-3-4 дерево.
- Всё, что знаем про B-деревья из лекции 2, здесь тоже верно: поиск, вставка с разбиением полного узла, удаление.
- Сегодня 2-3-4 нужно нам для другого: красно-чёрное дерево — это 2-3-4, записанное двоичным деревом.

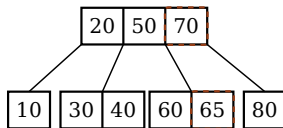
Вставка в 2-3-4: разбираем заранее

вставляем 65: узел на пути полон



70 вверх

разбили, потом вставили



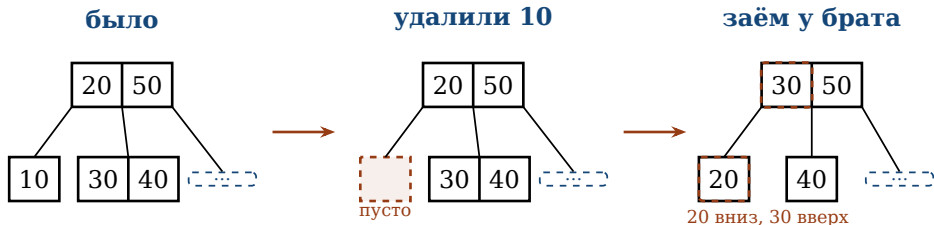
- Спускаемся к листу. Полный узел на пути (3 ключа) разбираем сразу: средний ключ уходит в родителя.
- В родителе место есть: полный родитель был бы разбит раньше.
- Высота растёт, только когда разбирают корень. Время — $O(\log n)$.

Удаление из 2-3-4: узел может опустеть

1. Ключ во внутреннем узле сначала меняем местами с соседним по порядку ключом — он лежит в листе. Так удаление всегда идёт из листа.
2. Удаляем ключ из листа. Было в листе 2 или 3 ключа — готово.
3. Был один ключ — лист **опустел**. Выбросить его нельзя: все листья должны оставаться на одной глубине.

Опустевший узел чинят одним из двух способов: **заём у брата** или **слияние с братом**.

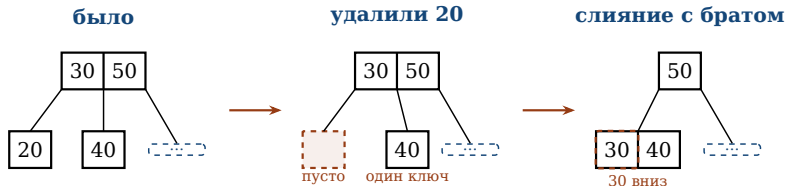
Заём: брат делится ключом через родителя



У брата $[30|40]$ два ключа — ему есть чем поделиться. Ключ 20 родителя спускается в пустой узел, ключ 30 брата поднимается на его место. Порядок ключей сохранён, глубина листов — тоже.

Ремонт окончен.

Слияние: брату нечем поделиться



- У брата один ключ. Пустой узел, ключ 30 родителя и брат 40 склеиваются в один узел $[30|40]$.
- У родителя стало на ключ меньше. Если он опустел — то же самое уровнем выше. Опустел корень — его убирают, дерево ниже на 1.
- На уровне — $O(1)$ работы, уровней — $O(\log n)$.

Красно-чёрные деревья: удаление

Красно-чёрное дерево — это 2-3-4, записанное двоичным деревом. Удаление из него — удаление из 2-3-4.

Четыре случая удаления вы уже видели

В лекции 2 удаление из красно-чёрного дерева было четырьмя случаями — по цвету брата и его детей (слайды 60–65).

Их можно выучить. Труднее понять, откуда они берутся.

Сегодня выведем их из 2-3-4: каждый случай — это заём или слияние, записанные цветами.

Красно-чёрное дерево = 2-3-4

Каждый узел 2-3-4 записываем маленьким двоичным деревом:

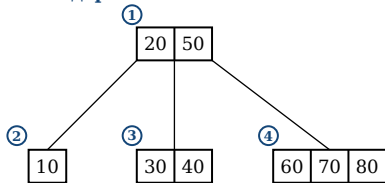
- 2-узел — один **чёрный** узел;
- 3-узел — чёрный узел и **красный** ребёнок;
- 4-узел — чёрный узел и два красных ребёнка.

Получается **красно-чёрное дерево**, дальше коротко — КЧ-дерево. Красный узел — не отдельный уровень, а ещё один ключ того же узла 2-3-4.

R. Sedgwick, «Left-leaning Red-Black Trees», 2008; в CLRS этого соответствия нет.

Одно и то же дерево: 2-3-4 и КЧ

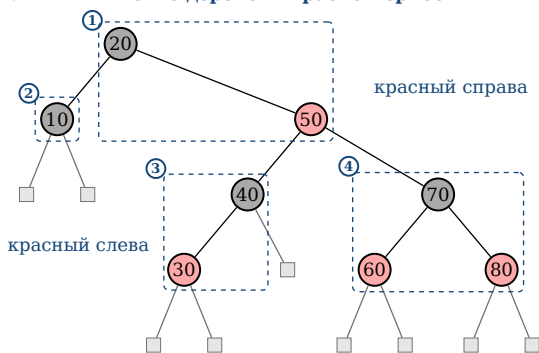
2-3-4 дерево



NIL — все листья на одной глубине



то же дерево — красно-чёрное



Номер в кружке — один и тот же узел. Наклон 3-узла — соглашение: узел 1 наклонён вправо, узел 3 — влево.

Свойства КЧ-дерева следуют из 2-3-4

1. **У красного узла нет красных детей.** Красный узел — ключ, прикрепленный к чёрному в том же узле 2-3-4. Его дети — уже другие узлы 2-3-4, а каждый узел 2-3-4 начинается с чёрного.
2. **На всех путях вниз одинаковое число чёрных узлов.** Чёрный узел — это узел 2-3-4, а в 2-3-4 все листья на одной глубине. Число чёрных на пути от корня до NIL (чёрная высота) — это высота 2-3-4 дерева.
3. **Высота $O(\log n)$.** Красных на пути не больше, чем чёрных, поэтому высота — не больше двух чёрных высот.

CLRS, гл. 13.1: свойства и лемма 13.1 (высота не больше $2 \log(n + 1)$).

Удаление из КЧ: первые шаги

1. У узла два ребёнка — копируем в него ключ предшественника (наибольший в левом поддереве) и удаляем узел предшественника. У того не больше одного ребёнка.
2. Удаляемый узел **красный** — просто убираем. В 2-3-4: из узла ушёл лишний ключ, узел не опустел.
3. Удаляемый узел **чёрный с красным ребёнком** — ребёнок встаёт на его место и становится чёрным. В 2-3-4: то же, узел не опустел.
4. Удаляемый узел **чёрный без детей** — его узел 2-3-4 **опустел**. Это единственный трудный случай.

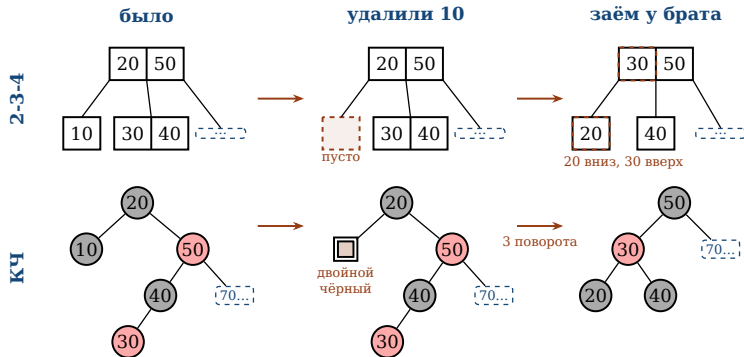
CLRS, гл. 13.4 (RB-Delete); CLRS берёт не предшественника, а преемника — разницы нет.

Двойной чёрный = опустевший узел 2-3-4

- Удалили чёрный лист. На его месте осталась пустая ссылка NIL. На пути через неё чёрных узлов **на один меньше**, чем на других путях.
- Чтобы не потерять счёт, эту ссылку временно считают чёрной дважды — **двойной чёрный**.
- В 2-3-4 это тот самый опустевший узел. Чиним так же: **заём у брата** или **слияние с братом**.

CLRS, гл. 13.4 («extra black», «doubly black»).

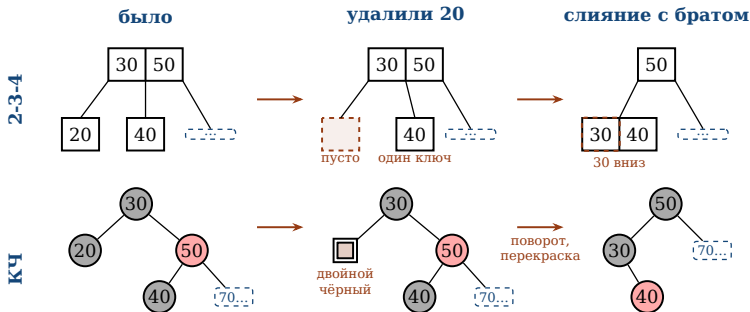
Заём в КЧ-записи: повороты и перекраска



В 2-3-4 брат отдал ключ через родителя. В двоичной записи это повороты и перекраска; после них чёрных на всех путях снова поровну.

КЧ-состояния получены процедурой RB-Delete из CLRS 13.4.

Слияние в КЧ-записи: брат краснеет



В 2-3-4 пустой узел, ключ 30 и брат 40 склеились. В КЧ-записи брат 40 краснеет — становится ключом узла 30. Если бы родитель опустел, двойной чёрный перешёл бы к нему, и ремонт пошёл бы уровнем выше.

КЧ-состояния получены процедурой RB-Delete из CLRS 13.4.

Удаление из КЧ — это удаление из 2-3-4

что видим в КЧ	что это в 2-3-4	что делаем
брат красный	брат — ключ из узла родителя	поворот; в 2-3-4 ничего не меняется
брат чёрный, есть красный ребёнок	брату есть чем поделиться	заём : 1-2 поворота и перекраска, стоп
брат чёрный, дети чёрные	брату нечем поделиться	слияние : брат краснеет; родитель красный — чернеет, стоп; чёрный — двойной чёрный переходит к нему
двойной чёрный — корень	опустел корень	снимаем: дерево ниже на 1

Поворотов — не больше трёх; вверх идут только перекраски, $O(\log n)$.

Живая демонстрация: визуализатор Galles

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

- Вставляем несколько ключей, потом удаляем по одному — и видим каждый шаг ремонта.
- Включаем **Show Null Leaves**: тогда видно NIL-листья, а двойной чёрный появляется как раз на NIL.
- Над деревом серой строкой визуализатор пишет, что делает сейчас. Что эти сообщения значат — на следующих двух слайдах.

David Galles, Data Structure Visualizations, University of San Francisco.

демо

Визуализатор: начало удаления

сообщение на экране

Node to delete has two children. Find largest node in left subtree.

Copy largest value of left subtree into node to delete.

Deleted node was red. No tree rotations required.

Coloring child of deleted node black

Coloring 'Null Leaf' double black

что это значит

у узла два ребёнка — ищем предшественника копируем ключ предшественника и удаляем его узел

удалили красный узел: в 2-3-4 узел не опустел

чёрный с красным ребёнком: ребёнок чернеет, узел не опустел

удалили чёрный лист: **двойной чёрный**, узел 2-3-4 опустел

Сообщения переписаны с экрана дословно, с опечаткой «children» из оригинала.

Визуализатор: ремонт

сообщение на экране	какое это правило
Double black node has red sibling. Rotate tree to make sibling black	брат красный: поворот, в 2-3-4 ничего не меняется
...black sibling ... nephew is red. One rotation can fix double-blackness. Double black node has black sibling and 2 black nephews. Push up black level	у брата красный ребёнок: заём
Pushing up black level created another double black node. Repeating	брату нечем поделиться: слияние
Double black node is root. Make it single black.	родитель был чёрным — опустел и он: ремонт уровнем выше опустел корень: дерево ниже на 1

nephew — племянник, ребёнок брата.

Как проверить КЧ-дерево

Ошибку в удалении глазами не найти. Нужна проверка, которая обходит всё дерево:

- корень чёрный;
- у красного узла нет красных детей;
- на всех путях от корня до NIL одинаковое число чёрных узлов.

Время — $O(n)$, один обход. Запускать после каждой операции в тестах.

В libstdc++ такая проверка есть: `_Rb_tree::__rb_verify()`.

libstdc++ (GCC 13), `bits/stl_tree.h`.

LLRB: КЧ-дерево с наклоном влево

- Куда наклонить 3-узел — соглашение (слайд 21). **LLRB** (left-leaning red-black tree, Седжвик, 2008) делает соглашение правилом: **красный ребёнок может быть только левым**.
- В варианте из учебника Седжвика «Algorithms» (4-е изд.) 4-узлов нет совсем: такое дерево соответствует 2-3 дереву, в узле 1 или 2 ключа.
- Зачем: случаев становится меньше, и код вставки получается очень коротким.

R. Sedgwick, «Left-leaning Red-Black Trees», 2008; R. Sedgwick, K. Wayne, «Algorithms», 4-е изд., §3.3.

LLRB: чем платят за короткий код

Вставка короче. Удаление дороже: на пути вниз оно делает повороты, и многие из них отменяются на пути вверх.

- Э. Колер: поворотов на одно удаление — 0.380 у обычного красно-чёрного дерева и 19.757 у LLRB, примерно в 52 раза больше. *по данным авторов, на их железе*
- Р. Браун, 2024: на 2^{18} ключах у LLRB при удалении в 25 раз больше поворотов, чем у красно-чёрного дерева с удалением снизу вверх; LLRB медленнее и при вставке, и при удалении. *по данным авторов, на их железе*

E. Kohler, «Left-Leaning Red-Black Trees Considered Harmful»; R. A. Brown, arXiv:2406.05162.

Удаление в деревьях: итог

дерево	ремонт после удаления	поворотов
AVL	пересчёт высот снизу вверх;	до $O(\log n)$
2-3-4	поворот может не вернуть высоту заём у брата или слияние; слияние может повториться выше	нет: переезжают ключи
КЧ	то же, что в 2-3-4, записанное цветами	не больше трёх
LLRB	как КЧ, но удаление делает лишние повороты	много

Время удаления везде — $O(\log n)$.

Что общего у всего, что мы делали

дерево	число про поддереву	как чинили после удаления
AVL	высота поддерева	пересчёт снизу вверх, пока высота меняется
2-3-4	глубина листьев — у всех одна	слияние уносит пустоту к родителю
КЧ	чёрная высота — на всех путях одна	двойной чёрный уходит к родителю

Во всех трёх деревьях в узле лежало число про поддереву, и удаление было его **ремонтom снизу вверх**.

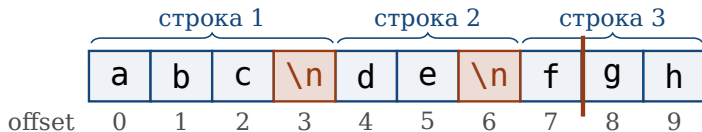
Дальше — то же, но число в узле полезно само по себе: сумма, минимум, количество.

Задача: редактор

Курсор — это смещение в байтах, а на экране — строка и колонка. Как переводить одно в другое быстро, если текст всё время меняется?

Курсор — это смещение, а на экране — строка и колонка

Редактор хранит текст как последовательность байт. Курсор — это **смещение** (offset): сколько байт стоит перед ним.



на экране:
строка 3,
колонка 2

курсор: offset 8

Строку и колонку надо пересчитывать **при каждом движении курсора**, а текст при этом всё время правят.

Наивно: считать переводы строк от начала — $O(n)$ на запрос

Чтобы узнать строку и колонку для смещения k :

1. пройти байты $0, 1, \dots, k - 1$ и сосчитать переводы строк;
2. строка = их число + 1;
3. колонка = $k - (\text{смещение начала этой строки}) + 1$.

Это $O(n)$ на **каждый** запрос, где n — длина текста. В файле на миллионы байт — миллионы шагов на каждое нажатие стрелки.

Посчитать заранее: запрос за $O(1)$

Заведём две таблицы и заполним их одним проходом по тексту:

- $\text{row}[i]$ — номер строки, в которой стоит байт i ;
- $\text{start}[r]$ — смещение, с которого начинается строка r .

Тогда для смещения k : строка $r = \text{row}[k]$, колонка $= k - \text{start}[r] + 1$.

Запрос — два чтения из массива, $O(1)$. Но что будет, когда текст поправят?

Одна вставка в начало меняет всю таблицу

было:

	a	b	c	\n	d	e	\n	f	g	h
offset	0	1	2	3	4	5	6	7	8	9
строка	1	1	1	1	2	2	2	3	3	3

стало: вставили \n в начало

	\n	a	b	c	\n	d	e	\n	f	g	h
offset	0	1	2	3	4	5	6	7	8	9	10
строка	1	2	2	2	2	3	3	3	4	4	4

- вставили один байт — **все** байты после него сдвинулись на одно смещение;
- вставили перевод строки — у **всех** следующих строк номер вырос на единицу.

Таблицы надо пересчитать целиком: $O(n)$ на каждую правку.

Номер строки — это сумма на отрезке

Поставим под каждым байтом число $b[i]$: 1, если это перевод строки, и 0 иначе.

текст	a	b	c	\n	d	e	\n	f	g	h
b	0	0	0	1	0	0	1	0	0	0
	0	1	2	3	4	5	6	7	8	9

$$b[0] + \dots + b[7] = 2 \Rightarrow \text{offset } 8 - \text{строка } 2 + 1 = 3$$

Строка для смещения $k = b[0] + \dots + b[k-1] + 1$. Заменить байт — значит поменять одно число $b[i]$.

Что нужно: и запрос, и правка за $O(\log n)$

	запрос	правка
считать от начала	$O(n)$	$O(1)$
посчитать всё заранее	$O(1)$	$O(n)$
нужно	$O(\log n)$	$O(\log n)$

Идея: разбить текст на куски и для каждого куска хранить готовую сумму b — число переводов строк в нём. Куски сложить в дерево: запрос собирает ответ из нескольких готовых кусков, правка чинит только те куски, где лежит изменённый байт.

Такое дерево — **дерево отрезков**. Сдвиг смещений при вставке оно не решает — к этому вернёмся в конце части.

Агрегат: одно значение на весь отрезок

Агрегат — значение, посчитанное по всем элементам отрезка одной операцией: сумма, минимум, максимум, НОД.

Отрезок 3, 1, 4, 1, 5:

$$\text{сумма} = 14 \quad \min = 1 \quad \max = 5 \quad \text{НОД} = 1$$

В задаче редактора агрегат — сумма b на отрезке: сколько переводов строк в куске текста.

Моноид: из чего можно собирать агрегат по кускам

Моноид — множество с операцией $\circ$, которая **ассоциативна**: $(x \circ y) \circ z = x \circ (y \circ z)$, и у которой есть **нейтральный элемент** e : $x \circ e = e \circ x = x$.

множество	операция	нейтральный
числа	+	0
числа	min	$+\infty$
числа	max	$-\infty$
неотрицательные целые	НОД	0
строки	склейка	пустая строка

Ассоциативность позволяет считать по кускам при любом разбиении отрезка; e — агрегат пустого отрезка.

Обратная операция не нужна — поэтому годятся $\min$ и $\max$

Префиксные суммы отвечают на запрос вычитанием:

$$a[l] + \dots + a[r] = \text{pref}(r) - \text{pref}(l - 1).$$

Нужна операция, обратная к $+$. Для $\min$ её нет: на массиве 1, 5, 3 минимум префикса $[1..3]$ равен 1, префикса $[1..1]$ тоже 1 — а минимум $[2..3]$ равен 3, и из двух единиц его не получить.

Дерево отрезков ничего не вычитает: оно только складывает готовые куски операцией моноида. Поэтому $\min$, $\max$ и НОД работают в нём так же, как сумма.

Дерево отрезков

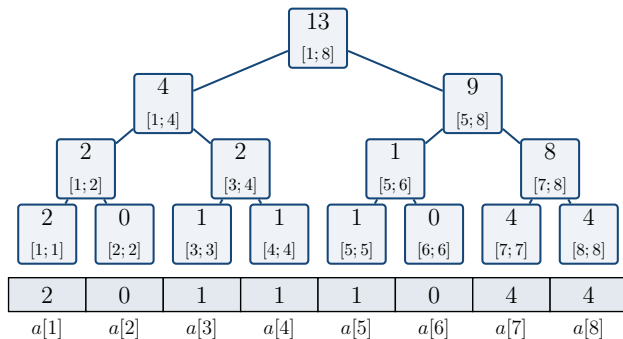
Определение, запрос за $O(\log n)$ и почему так,
правка, раскладка в массиве: $4n$ и $2n$.

Определение

Дерево отрезков над массивом $a[1..n]$ — двоичное дерево, в котором каждый узел отвечает за отрезок массива и хранит агрегат по нему.

- корень — отрезок $[1; n]$;
- у узла $[l; r]$, $l < r$, два ребёнка: $[l; m]$ и $[m+1; r]$, где $m = \lfloor (l + r)/2 \rfloor$;
- лист — отрезок из одного элемента;
- агрегат узла = агрегат левого ребёнка $\circ$ агрегат правого.

Дерево отрезков над $a[1..8]$, агрегат — сумма



В узле — сумма по отрезку, под ней — сам отрезок. Каждый уровень делит отрезки пополам, поэтому высота — $\lceil \log_2 n \rceil$; здесь 3. Узлов — $2n - 1 = 15$.

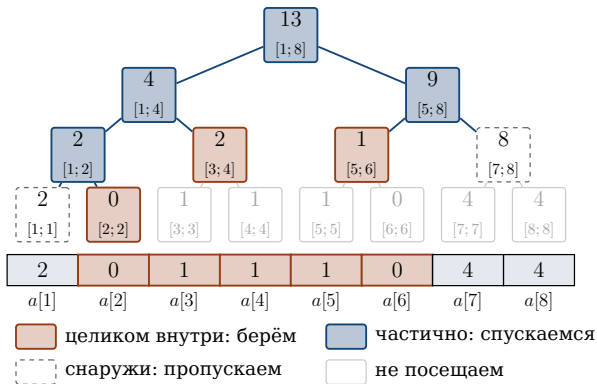
Запрос $query(l, r)$: три случая

Считаем агрегат на $[l; r]$. Начинаем с корня; для узла с отрезком $[tl; tr]$:

1. $[tl; tr]$ **целиком внутри** $[l; r]$ — вернуть значение узла, вниз не идти;
2. $[tl; tr]$ **целиком снаружи** $[l; r]$ — вернуть нейтральный элемент e , вниз не идти;
3. пересекаются **частично** — спросить обоих детей и сложить их ответы операцией $\circ$.

Ответ собирается из узлов первого случая: каждый — готовый кусок отрезка $[l; r]$.

Запрос $\text{sum}(2, 6)$ на картинке



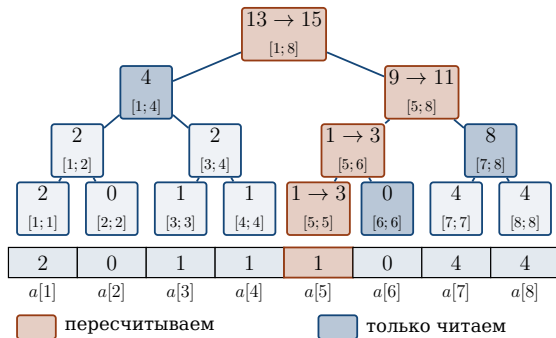
$\text{sum}(2, 6) = [2; 2] + [3; 4] + [5; 6] = 0 + 2 + 1 = 3$ — три узла вместо пяти элементов.

Почему запрос — $O(\log n)$

1. Узел пересекает $[l; r]$ **частично**, только если внутри его отрезка проходит граница запроса: левая (между $l - 1$ и l) или правая (между r и $r + 1$).
2. Отрезки узлов одного уровня не пересекаются. Значит, каждую границу содержит не больше одного узла уровня, и **частичных узлов на уровне не больше двух**.
3. Спускаемся мы только из частичных узлов. Значит, на следующем уровне посещаем не больше $2 \cdot 2 = 4$ узлов.
4. Уровней $\lceil \log_2 n \rceil + 1$. Всего посещённых узлов — не больше $4(\lceil \log_2 n \rceil + 1) = O(\log n)$.

На картинке $\text{sum}(2, 6)$ частичных узлов по уровням: 1, 2, 1, 0.

Правка: пересчитать путь от листа к корню



$a[5] := 3$. Меняем лист, затем поднимаемся: каждый узел на пути пересчитываем из двух детей. Узлов на пути — высота $+1$, то есть $O(\log n)$; остальные узлы агрегат не меняют.

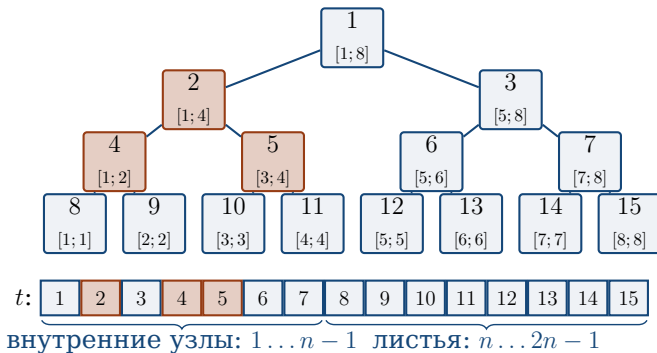
Код: рекурсивный запрос и правка (агрегат — сумма)

```
74     long long query(int v, int tl, int tr, int l, int r) const {
75         if (l > r) return 0;
76         if (l == tl && r == tr) return t[v];
77         int tm = (tl + tr) / 2;
78         return query(v * 2, tl, tm, l, std::min(r, tm)) +
79                query(v * 2 + 1, tm + 1, tr, std::max(l, tm + 1), r);
80     }
81
95     void update(int v, int tl, int tr, int pos, long long val) {
96         if (tl == tr) { t[v] = val; return; }
97         int tm = (tl + tr) / 2;
98         if (pos <= tm) update(v * 2, tl, tm, pos, val);
99         else update(v * 2 + 1, tm + 1, tr, pos, val);
100        t[v] = t[v * 2] + t[v * 2 + 1];
101    }
```

Запрос обрезает $[l; r]$ по отрезку ребёнка: «снаружи» — это $l > r$, ответ 0; «внутри» — совпадение с отрезком узла.

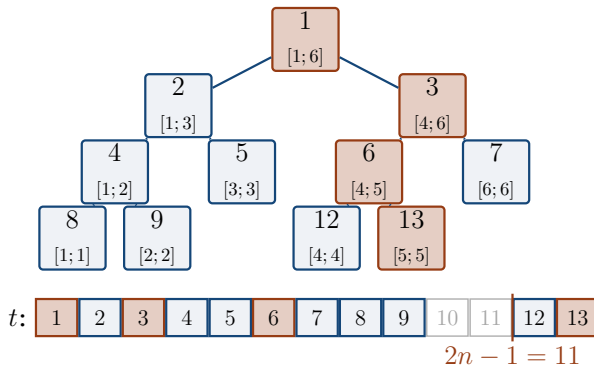
cp-algorithms, «Segment Tree» — тот же код; массив индексируется с нуля.

Дерево в массиве: узел v — ячейка v , дети — $2v$ и $2v + 1$



Указатели не нужны: номер ребёнка вычисляется. При $n = 8$ узлы занимают ячейки $1 \dots 15$, то есть массива из $2n$ ячеек хватает (ячейка 0 пустая).

Если n — не степень двойки, $2n$ ячеек мало



$n = 6$: спуск $1 \rightarrow 3 \rightarrow 6 \rightarrow 13$ приходит в ячейку 13, а в массиве из $2n$ ячеек последняя — 11. Ячейки 10 и 11 при этом пустые.

Откуда $4n$: сколько ячеек хватает всегда

Почему хватает. Глубина дерева — не больше $\lceil \log_2 n \rceil$. У узла на глубине d номер меньше 2^{d+1} . Значит, любой номер меньше $2^{\lceil \log_2 n \rceil + 1}$, а $2^{\lceil \log_2 n \rceil} < 2n$ — то есть номер меньше $4n$.

Проверка перебором по всем $n = 1 \dots 4096$ (наибольший номер узла при такой раскладке):

- $2n$ ячеек не хватает в 4062 случаях из 4096;
- $4n$ ячеек хватает всегда;
- худший случай — $n = 2080$: номер 8065, это 0.97 от $4n$.

Отсюда в коде `t(4 * n)`: запас не лишний, $2n$ почти всегда мало.

Перебор — наш, проверен двумя независимыми обходами.

Снизу вверх: $2n$ ячеек и никакой рекурсии

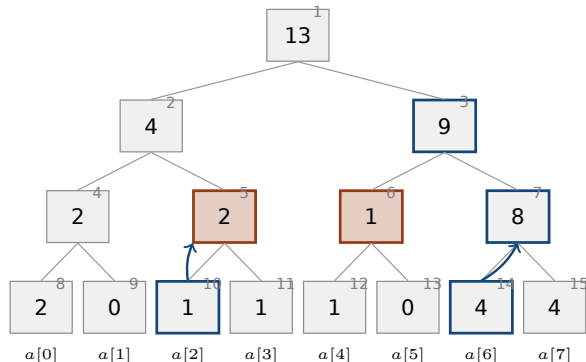
Другая раскладка того же дерева (массив индексируется с нуля):

- элемент $a[i]$ лежит в ячейке $n + i$: листья — ячейки $n \dots 2n - 1$;
- родитель ячейки k — ячейка $\lfloor k/2 \rfloor$, дети ячейки k — $2k$ и $2k + 1$;
- внутренние узлы — ячейки $1 \dots n - 1$; всего $2n$ ячеек.

Запрос на $[l, r]$: поставить l и r на листья и подниматься обоими к корню. Если l — правый ребёнок (нечётный), его родитель захватил бы элемент левее l : берём $t[l]$ и сдвигаем l вправо. Для r — симметрично. **Правка:** записать лист и пересчитать всех предков. Обе операции — $O(\log n)$, циклом.

При n не степени двойки это уже несколько деревьев, но код остаётся верным.

Снизу вверх на картинке: запрос $[2, 6)$ при $n = 8$



$l = 2 + 8 = 10, r = 6 + 8 = 14.$

Шаг 1. $l = 10$ и $r = 14$ чётные: ничего не берём. Поднимаемся: $l = 5, r = 7.$

Шаг 2. $l = 5$ нечётный — берём $t[5] = 2$ (это $a[2..3]$), $l = 6$. $r = 7$ нечётный — $r = 6$, берём $t[6] = 1$ (это $a[4..5]$). Поднимаемся: $l = r = 3.$

Шаг 3. $l = r$ — стоп.

Ответ: $2 + 1 = 3 = a[2] + \dots + a[5].$

Код — на следующем слайде; раскладка — как у Слотина.

Снизу вверх: код

```
116     long long query(int l, int r) const {
117         long long s = 0;
118         for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
119             if (l & 1) s += t[l++];
120             if (r & 1) s += t[--r];
121         }
122         return s;
123     }
124     void update(int pos, long long val) {
125         for (t[pos += n] = val; pos > 1; pos >>= 1) t[pos >> 1] = t[pos] + t[pos ^ 1];
126     }
```

query — сумма на $[l, r)$. $\text{pos} \ ^\wedge \ 1$ — второй ребёнок того же родителя.

Скорость рекурсивной раскладки $4n$ и этой, $2n$ (наш замер, AMD Zen 5, одно ядро, gcc -O2): $4n$ — 276.60 нс, $2n$ — 44.89 нс на запрос при $n = 2^{20}$, в 6.2 раза.

Код — по Слотину, «Segment Trees».

Что такое AtCoder Library и зачем она здесь

- **AtCoder** — японская платформа соревнований по программированию.
- **AtCoder Library (ACL)** — её официальная открытая библиотека алгоритмов на C++ (с 2020 года): дерево отрезков, Фенвик, графы, строки. На соревнованиях AtCoder её можно подключать.
- **Зачем нам.** Это аккуратно написанная, общедоступная реализация дерева отрезков. Посмотрим, что ей нужно знать об операции, — ровно моноид.

github.com/atcoder/ac-library.

Библиотека AtCoder: моноид в параметрах шаблона

23 `template <class S, S (*op)(S, S), S (*e)()> struct segtree {`

S — множество, op — ассоциативная операция, e — нейтральный элемент. Это ровно моноид. Пример — минимум:

```
int op(int x, int y) { return std::min(x, y); }
int e() { return INT_MAX; }
atcoder::segtree<int, op, e> seg(a);    // a: std::vector<int>
seg.set(p, x);                         // a[p] = x
int m = seg.prod(l, r);                // min on [l, r)
```

Дерево не знает, что считает минимум: поменяйте op и e — получится сумма, максимум или НОД.

AtCoder Library, [atcoder/segtree.hpp](#), [github.com/atcoder/ac-library](#).

«Source: folklore»: у дерева отрезков нет автора

```
1 /**
2  * Author: Lucian Bicsi
3  * Date: 2017-10-31
4  * License: CC0
5  * Source: folklore
```

- В KACTL источник записан как «folklore», в e-maxx и cp-algorithms автора нет вовсе.
- Не путать: «segment tree» Бентли (1977) — другая, геометрическая структура с тем же названием.

KACTL, `content/data-structures/SegmentTree.h` (CC0); J. L. Bentley, «Solutions to Klee's rectangle problems», CMU, 1977 (не опубликовано).

Чего дерево отрезков не умеет: вставить символ в середину

Умеет — для любого моноида, за $O(\log n)$:

- агрегат на любом отрезке;
- замену одного элемента.

Для редактора это значит: номер строки для любого смещения — за $O(\log n)$, даже если байт заменили на перевод строки.

Не умеет — менять длину массива. Дерево построено над $a[1..n]$ с фиксированным n : элемент i навсегда лежит в своём листе.

Вставили символ в середину текста — все следующие элементы сдвинулись, и дерево надо строить заново, $O(n)$.

К вставке вернёмся: нужно дерево, в котором позиция элемента не зашита в раскладку.

Дерево Фенвика

Префиксные суммы в n ячейках — и кто придумал это первым.

lowbit(i) — младший установленный бит числа i

Запишем i в двоичной системе и оставим только самую правую единицу. Получившееся число и есть lowbit(i).

i	двоичная запись	lowbit(i)
12	1 1 00	100 = 4
6	1 1 0	10 = 2
8	1 000	1000 = 8
7	11 1	1 = 1

lowbit(i) — всегда степень двойки. У нечётного i он равен 1.

Почему $\text{lowbit}(i) = i \& (-i)$

В дополнительном коде $-i$ — это «инвертировать все биты i и прибавить 1». Возьмём $i = 12$ в восьми битах:

$i = 12$	0000 1100
инвертировали	1111 0011
прибавили 1: $-i$	1111 0100
$i \& (-i)$	0000 0 1 00 = 4

Нули справа после инверсии стали единицами, и $+1$ переносится через них ровно до младшей единицы i . Левее неё биты i и $-i$ противоположны, правее — оба нули. Общая у них только эта единица.

Определение: дерево Фенвика

Дерево Фенвика для массива $a[1..n]$ — это массив $t[1..n]$, где

$$t[i] = a[i - \text{lowbit}(i) + 1] + \dots + a[i].$$

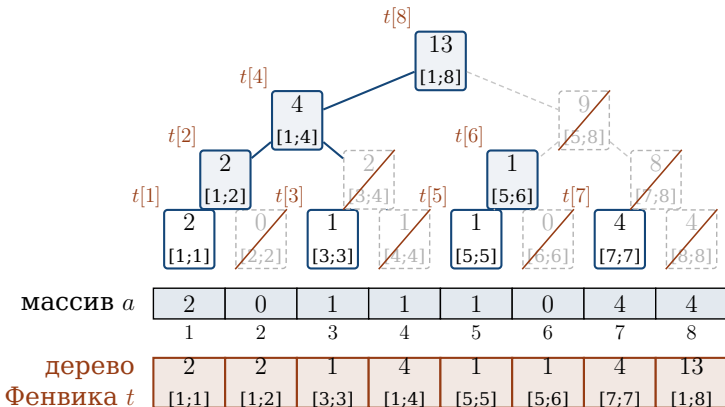
- ячейка i хранит сумму $\text{lowbit}(i)$ элементов, последний из которых — $a[i]$;
- ячеек столько же, сколько элементов: n ;
- задача — префиксная сумма $\text{pref}(i) = a[1] + \dots + a[i]$ и правка одного элемента, обе за $O(\log n)$.

Пример на восьми элементах

$$a[1..8] = 2, 0, 1, 1, 1, 0, 4, 4$$

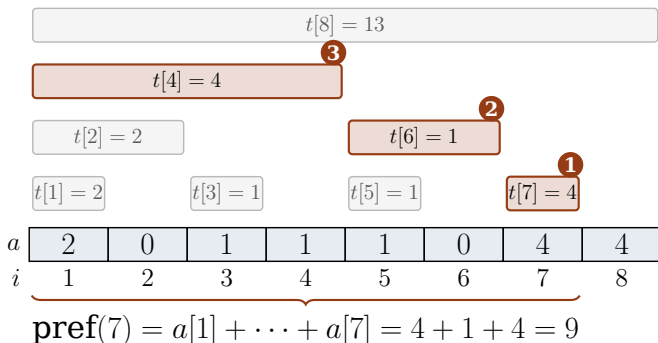
i	двоичная запись	lowbit(i)	$t[i]$ хранит	значение
1	0001	1	$a[1]$	2
2	0010	2	$a[1..2]$	2
3	0011	1	$a[3]$	1
4	0100	4	$a[1..4]$	4
5	0101	1	$a[5]$	1
6	0110	2	$a[5..6]$	1
7	0111	1	$a[7]$	4
8	1000	8	$a[1..8]$	13

Это дерево отрезков без правых детей



Остались корень и все левые дети — ровно 8 узлов из 15. Узел с отрезком $[l; i]$ лежит в ячейке $t[i]$.

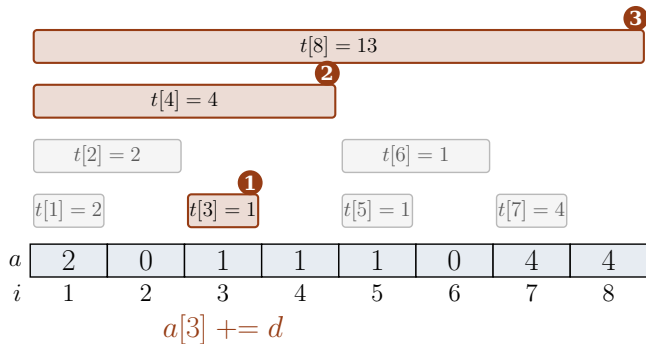
Префиксная сумма: $i \rightarrow i - \text{lowbit}(i)$



$7 = 111 \rightarrow 6 = 110 \rightarrow 4 = 100 \rightarrow 0$: складываем $t[7] + t[6] + t[4]$.

Почему $O(\log n)$: шаг стирает одну единицу в записи i , а их не больше $\log_2 n + 1$.

Правка элемента: $i \rightarrow i + \text{lowbit}(i)$



$3 = 0011 \rightarrow 4 = 0100 \rightarrow 8 = 1000 \rightarrow 16 > n$: правим $t[3]$, $t[4]$, $t[8]$ — все отрезки, где есть 3.

Почему $O(\log n)$: $\text{lowbit}(i)$ с каждым шагом хотя бы удваивается и не больше n .

Обе операции в коде

```
int n, t[N + 1];           // t[1..n], t[0] unused

void add(int i, int d) {    // a[i] += d
    for (; i <= n; i += i & -i) t[i] += d;
}

int pref(int i) {          // a[1] + ... + a[i]
    int s = 0;
    for (; i > 0; i -= i & -i) s += t[i];
    return s;
}
```

Вся «математика» — в выражении $i \& -i$. Индексы с единицы: при $i = 0$ цикл правки никогда бы не кончился.

Чего дерево Фенвика не умеет

Сумма на отрезке — через вычитание двух префиксов:

$$a[l] + \dots + a[r] = \text{pref}(r) - \text{pref}(l - 1).$$

На примере: $a[3..6] = \text{pref}(6) - \text{pref}(2) = 5 - 2 = 3$.

Минимум на отрезке так не получить: у минимума нет вычитания. Минимум на $[1; 5]$ равен 0, на $[1; 2]$ — тоже 0; отсюда не узнать, что на $[3; 5]$ он равен 1.

Для минимума и максимума нужно дерево отрезков: там правые дети на месте.

Откуда он взялся: сжатие данных, 1994

Питер Фенвик придумал эту структуру для **сжатия данных**. Арифметическому кодированию на каждом символе текста нужны обе операции:

- сколько раз в прочитанном тексте встретились символы, стоящие в алфавите раньше текущего, — префиксная сумма частот;
- частота текущего символа выросла на 1 — правка.

«A major cost in adaptive arithmetic data compression is the maintenance of the table of cumulative frequencies» — первая фраза статьи.

Само сжатие — в следующем семестре.

Рябко, 1989: та же задача на пять лет раньше

Борис Рябко хранит, **сколько раз встретился каждый из L символов алфавита**, и ещё суммы пар соседних символов, четвёрок, восьмёрок — «всего $L + L/2 + L/4 + \dots = 2L - 1$ чисел».

все восемь	13							
четвёрки	4				9			
пары	2		2		1		8	
каждый символ	2	0	1	1	1	0	4	4
	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8

$a_1, \dots, a_7$ встретились $4 + 1 + 4 = 9$ раз

Сколько раз встретились $a_1, \dots, a_7$: сложили три хранимых числа. В общем случае их не больше $\log_2 L$.

Рябко: пришёл ещё один символ



Частота a_5 выросла на 1 — значит, на 1 выросли все суммы, в которые она входит: по одной на каждом уровне, всего 4 числа из 15. В общем случае — $\log_2 L + 1$, то есть $O(\log L)$.

Одна идея — две раскладки памяти

	Рябко, 1989	Фенвик, 1994
что хранится	суммы всех пар, четвёрок, восьмёрок...	сумма $\text{lowbit}(i)$ элементов, кончающихся на i
сколько чисел	$2L - 1$	n
запрос и правка	$O(\log L)$	$O(\log n)$

Раскладка Рябко — это дерево отрезков из части 2. Выкиньте из него правых детей — останутся ровно n ячеек Фенвика.

Фенвик статью Рябко не цитирует: одну идею придумали дважды.

Цена записи: дерево против префиксных сумм

Массив из 65 536 нулей и единиц, в нём поменяли один элемент. Нужно снова уметь отвечать на префиксные суммы.

- **Дерево Фенвика:** одна правка, $O(\log n)$ ячеек.
- **Массив префиксных сумм** (по блокам из 64 элементов, 1024 числа): пересчитать весь заново.

правка дерева: 14.64 нс
перестроение: 214.28 нс
разница: в 14.6 раза

Ради этого дерево и нужно: запись в нём дешёвая.

Наш замер: AMD Zen 5, одно ядро, gcc -O2.

Декартово дерево

Дерево поиска по ключу и куча по приоритету:
split, merge и почему это быстро.

Зачем декартово дерево: баланс без правил

- Задача та же, что у AVL и красно-чёрного дерева: множество ключей, поиск, вставка и удаление за $O(\log n)$.
- У обычного дерева поиска форма зависит от порядка вставки: ключи по возрастанию выстраиваются в цепочку длины n .
- AVL и красно-чёрное держат баланс правилами — утром мы видели, сколько там случаев.
- Декартово дерево вместо правил берёт случайность: каждому ключу — случайный приоритет, и дерево в среднем получается таким же, как дерево поиска из случайно перемешанных ключей. Глубина — $O(\log n)$ в среднем, при любых данных.

Зачем split и merge

Всё декартово дерево собирается из двух операций:

- split — разрезать дерево на две части по ключу;
- merge — склеить две части обратно.

Вставка — split и два merge; удаление — два split и merge.
И операции с целыми кусками: вырезать отрезок,
переставить его, объединить два множества.

В AVL и красно-чёрном дереве резать и склеивать тоже можно, но код сложный; здесь это две короткие функции.
Этим воспользуется неявное декартово дерево.

Blelloch, Reid-Miller, «Fast Set Operations Using Treaps», 1998.

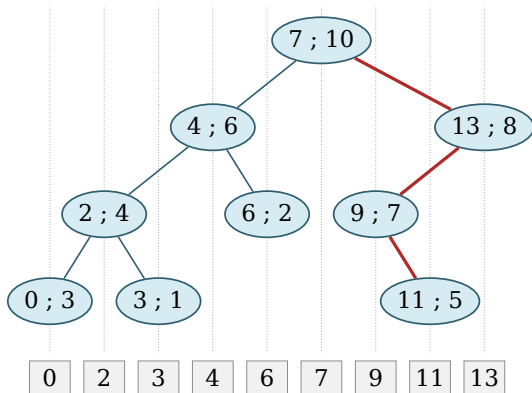
Декартово дерево (treap): определение

Декартово дерево (treap) — двоичное дерево, в каждом узле которого лежит пара (ключ, приоритет), и при этом:

- по ключам это **дерево поиска**: в левом поддереве ключи меньше, в правом — больше;
- по приоритетам это **куча**: у родителя приоритет больше, чем у детей.

Ключи — наши данные. Приоритет выбирают случайно, когда ключ вставляют.

Пример: девять пар (ключ ; приоритет)



В узле записано: ключ ; приоритет.

Дерево поиска. Внизу — ключи в порядке обхода «левое поддерево, узел, правое поддерево»: 0, 2, 3, 4, 6, 7, 9, 11, 13 — по возрастанию.

Куча. Красный путь: приоритеты 10, 8, 7, 5 убывают сверху вниз. Так на любом пути от корня вниз.

Пример — из слайдов Н. Макарова «Декартово дерево» (2017), без двух листьев.

Набор пар задаёт дерево однозначно

Пусть все приоритеты разные. Тогда дерево строится так:

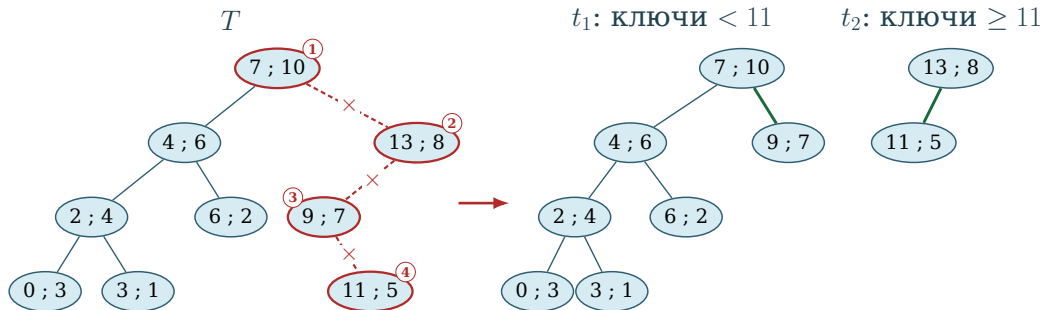
1. корень — пара с наибольшим приоритетом (так требует куча);
2. пары с ключом меньше корневого — в левом поддереве, с большим — в правом (так требует дерево поиска);
3. каждое поддерево строится по тому же правилу.

Выбора нигде нет — значит, дерево одно. Оно не зависит от того, в каком порядке вставляли ключи.

У обычного дерева поиска форма от порядка зависит: ключи, вставленные по возрастанию, выстраиваются в цепочку глубины n . На практике приоритеты — случайные 64-битные числа, совпадения практически исключены.

split: одно дерево — два

$\text{split}(T, x)$ получает декартово дерево T и число x и возвращает два декартовых дерева: t_1 — все ключи $< x$, t_2 — все ключи $\geq x$.



Пример: $x = 11$. Разрезаны только рёбра на пути спуска (красные), зелёные рёбра — новые. Остальное дерево не тронуто.

split: как работает и сколько стоит

Правило. Смотрим на корень t .

- t пусто: ответ — два пустых дерева.
- ключ $t < x$: t с левым поддеревом — в t_1 . Правое поддерево режем тем же split на (a, b) ; правым ребёнком t ставим a . Ответ: (t, b) .
- ключ $t \geq x$: зеркально — режем левое, левым ребёнком t ставим b . Ответ: (a, t) .

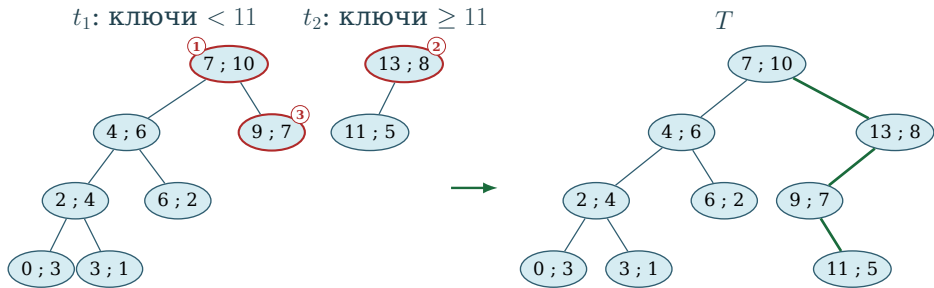
Пример, $x = 11$:

1. $7 < 11$: 7 в t_1 , вправо;
2. $13 \geq 11$: 13 в t_2 , влево;
3. $9 < 11$: 9 в t_1 , вправо;
4. $11 \geq 11$: 11 в t_2 , влево; пусто.

Время: один уровень за шаг — $O(\text{глубины } T)$.

merge: два дерева — одно

$\text{merge}(t_1, t_2)$: все ключи t_1 меньше всех ключей t_2 . Результат — одно декартово дерево из всех их пар. Это операция, обратная split .



Склеили те же две половины — и получили исходное T : набор пар тот же, а дерево по набору одно.

merge: как работает и сколько стоит

Правило. Сравниваем приоритеты корней.

- одно дерево пусто: ответ — другое;
- у корня t_1 приоритет больше: он — корень ответа, его правым ребёнком ставим $\text{merge}(\text{правое } t_1, t_2)$;
- иначе корень ответа — корень t_2 , его левым ребёнком ставим $\text{merge}(t_1, \text{левое } t_2)$.

Время: шаг — уровень в t_1 или в t_2 , всего $O(\text{глубина } t_1 + \text{глубина } t_2)$.

Пример:

1. $10 > 8$: корень — 7;10;
2. $7 < 8$: ниже — 13;8;
3. $7 > 5$: ниже — 9;7, справа у него пусто: туда 11;5.

Почему глубина в среднем $O(\log n)$

Зачем это знать: split и merge стоят $O(\text{глубины})$, а у дерева-цепочки глубина n .

- Вставим ключи в обычное дерево поиска по убыванию приоритета — получится ровно наше декартово дерево.
- Приоритеты случайны, значит, этот порядок — случайная перестановка ключей.
- У дерева поиска из случайной перестановки средняя глубина узла — $O(\log n)$.

Итог: в среднем $O(\log n)$ при любых ключах и любом порядке вставки. Случайны приоритеты, а не данные.

Aragon, Seidel, FOCS 1989; Кормен и др., «Алгоритмы: построение и анализ», 3-е изд., § 12.4.

Лемма: кто чей предок

Ключи по возрастанию: $x_1 < x_2 < \dots < x_n$; приоритеты случайны и различны.

Лемма. x_i — предок $x_j \iff$ у x_i наибольший приоритет среди $x_i, \dots, x_j$.

Это случается с вероятностью $1/(|i - j| + 1)$. Сложим по всем i :

средняя глубина $x_i = H_i + H_{n-i+1} - 2 \leq 2 \ln n$, $H_k = 1 + \frac{1}{2} + \dots + \frac{1}{k}$.

Зачем лемма: она даёт точную среднюю глубину каждого ключа — отсюда $O(\log n)$.

Aragon, Seidel, «Randomized Search Trees», FOCS 1989 — там же доказательство.

Откуда названия

- **«Декартово дерево»** (Cartesian tree) — Жан Вюльмен, 1980. Каждый узел — точка на плоскости: ключ — координата x , приоритет — координата y . Координаты декартовы, то есть названы в честь Декарта.
- **treap** = **tree** + **heap**, дерево + куча. Слово — из статьи Сесилии Арагон и Раймунда Зайделя, 1989: в ней приоритеты выбираются случайно.
- **Сесилия Арагон** — профессор Вашингтонского университета, первая латиноамериканка в сборной США по высшему пилотажу.

J. Vuillemin, «A Unifying Look at Data Structures», CACM 23(4), 1980; C. Aragon, R. Seidel, FOCS 1989; C. Aragon, «Flying Free», 2020.

Неявное декартово дерево

Ключей нет: номер элемента считается по размерам поддеревьев.

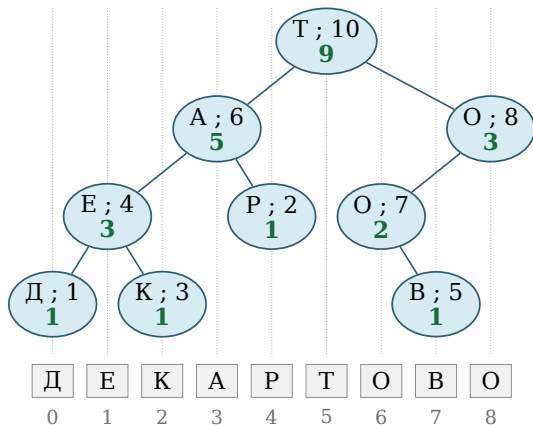
Неявное декартово дерево: определение

Неявное декартово дерево хранит последовательность элементов — например, символы текста.

- В узле — элемент, случайный приоритет (по нему это куча, как раньше) и **размер поддерева**.
- **Ключ есть, но нигде не записан:** ключ элемента — его номер в последовательности, его вычисляют при спуске по размерам поддеревьев. Отсюда название: дерево «по неявному ключу».
- Порядок элементов — порядок обхода «левое поддерево, узел, правое поддерево».

Хранить номера нельзя — вставка в начало поменяла бы их все. В лекции 2 неявными были указатели, сегодня — ключ.

Пример: строка «ДЕКАРТОВО»



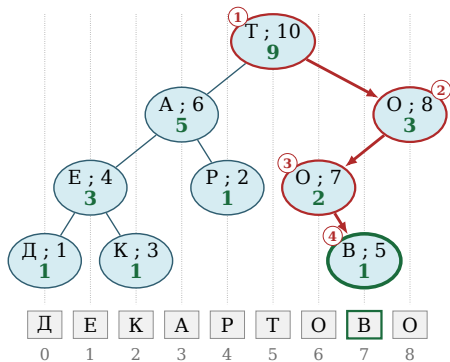
В узле: буква ; приоритет, под ними **зелёным** — размер поддерева.

Внизу — строка в порядке обхода, под буквами — номера позиций, с нуля.

Размер узла = размер левого + размер правого + 1. У корня: $5 + 3 + 1 = 9$.

Скелет и приоритеты — те же, что у дерева с ключами.

Как найти элемент с номером k



L — размер левого поддерева; он записан в левом ребёнке (пусто — $L = 0$).

$k < L$: влево, k не меняется;

$k = L$: нашли;

$k > L$: вправо, $k := k - L - 1$.

Пример, $k = 7$:

1. $L = 5$, $7 > 5$: вправо, $k = 1$;

2. $L = 2$, $1 < 2$: влево;

3. $L = 0$, $1 > 0$: вправо, $k = 0$;

4. $L = 0 = k$: нашли «В».

Время: $O(\text{глубины}) = O(\log n)$ в среднем.

split и merge по номеру

$\text{split}(T, k)$: t_1 — первые k элементов, t_2 — остальные. Спуск тот же, что при поиске:

- $L \geq k$ — корень идёт в t_2 , режем левое поддереву по k ;
- $L < k$ — корень идёт в t_1 , режем правое поддереву по $k - L - 1$.

merge — тот же, что раньше: на ключи он и так не смотрел.

Одно добавление: когда у узла сменился ребёнок, его размер пересчитывают: размер левого + размер правого + 1. Это $O(1)$ на каждый узел пути, так что обе операции — по-прежнему $O(\text{глубины})$.

«Вставить в середину»

Задача: вставить элемент на позицию k ; все элементы с позиции k и дальше сдвигаются на одну вправо.

- **Массив:** сдвинуть весь хвост — $O(n)$.
- **Дерево отрезков:** не умеет. Позиция — это номер листа, она зашита в раскладку; вставка — перестройка, $O(n)$.
- **Неявное декартово:** `split` по k , `merge` левой части с новым узлом, `merge` с правой — в среднем $O(\log n)$. Номера в хвосте сдвинулись сами: их никто не хранил.

Массива под деревом нет. Последовательность хранится только как дерево; её порядок — порядок обхода. `split` не копирует элементы, а разрезает дерево по пути спуска, `merge` так же сшивает: меняется $O(\log n)$ указателей и размеров.

Это ровно то, что нужно редактору: вставить символ в середину текста.

Вставка — две строки кода

```
split(t, where[i], L, R);  
t = merge(merge(L, new Node(val[i], splitmix64(st))), R);
```

where[i] — позиция, val[i] — новый элемент, splitmix64 — генератор псевдослучайных чисел: это приоритет нового узла.

вставка в начало, $n = 2^{16}$: неявное дерево против std::vector — 44.70 нс против 3133.60 нс на одну вставку: в 70 раз

Наш код и наш замер; стенд — AMD Zen 5, одно ядро, gcc -O2.

Rope и редактор Zed

Задача из части 2 — перевести смещение в «строка : колонка», пока текст меняется, — решается тем, что мы уже построили.

Что у нас уже есть

- **Задача (часть 2).** Курсор — смещение (offset) от начала текста. На экране нужно показать строку и колонку. Перевод нужен на каждое движение курсора, а текст всё время правят.
- **Дерево отрезков (часть 2)** хранит в узле агрегат по отрезку и отвечает за $O(\log n)$ — но длина массива у него фиксирована: вставить символ в середину оно не умеет.
- **Неявное декартово дерево (часть 4)** умеет вставлять в середину за $O(\log n)$: позиция элемента вычисляется при спуске по размерам поддеревьев.

Соединим: неявное дерево + агрегат в узле.

Определение: Rope

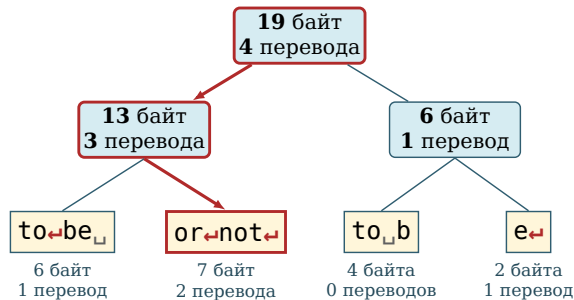
Rope — сбалансированное дерево для хранения текста.

- В **листьях** лежат короткие куски текста. Весь текст — это листья, прочитанные слева направо.
- В **каждом узле** лежит сводка по его поддереву: длина текста в байтах и число переводов строк.
- Сводка узла = сумма сводок его детей.

Rope — это неявное дерево из части 4 (позиция находится по длинам поддеревьев) плюс агрегат из части 2 (в узле — сумма по поддереву).

Boehm, Atkinson, Plass, «Ropes: an Alternative to Strings», 1995.

Роупе на примере: четыре строки, четыре листа



Весь текст — листья слева направо: to←be_or←not←to←be←

В узле — длина и число переводов строк поддеревя. ← — перевод строки, _ — пробел. Красным — спуск за offset = 11: буква «t» в слове «not».

По мотивам схем из поста «Rope & SumTree» (Zed Blog, 2024); пример и рисунок — наши.

Смещение → строка: спуск по сводкам

Правило в узле. Пусть у левого ребёнка длина L и s переводов строк. Если $\text{offset} < L$ — идём влево. Иначе идём вправо, а к ответу прибавляем s и из offset вычитаем L .

Пример: $\text{offset} = 11$ в тексте с прошлого слайда.

1. Корень: у левого ребёнка $L = 13$; $11 < 13$ — влево.
2. Узел $(13, 3)$: у левого ребёнка $L = 6$, $s = 1$; $11 \geq 6$ — вправо; строка $= 1$, $\text{offset} = 11 - 6 = 5$.
3. Лист `or not`: до позиции 5 один перевод строки — строка $= 1 + 1 = 2$. Последний перевод — в позиции 2, строка начинается с 3, колонка $= 5 - 3 = 2$.

Ответ: строка 2, колонка 2 — «t» в «not» (с нуля; на экране — 3:3).
Время — высота дерева, $O(\log n)$, плюс работа внутри одного листа.

Вставка в середину текста

Вставить кусок текста s по смещению k — так же, как в неявном декартовом дереве:

1. `split` по смещению k : дерево текста до k и дерево текста после; спуск — по длинам, как на прошлом слайде;
2. `merge`(левое, дерево из s), затем `merge` с правым;
3. сводки чинятся только на путях, по которым шли `split` и `merge`.

Каждый шаг — $O(\log n)$. Смещения остальных символов нигде не хранятся, поэтому ничего сдвигать не надо: они снова получатся при спуске.

Сравните с массивом «число переводов до позиции i » из части 2: там одна вставка стоила $O(n)$.

Редактор Zed: буфер — Rope на B+-дереве

- Zed — редактор кода. Текст файла в нём хранится как rope.
- Дерево — SumTree, это **B+-дерево**: B-дерево, в котором текст лежит только в листьях. У узла от 6 до 12 детей, лист — кусок текста до 128 байт.
- В каждом узле — сводка TextSummary: длина, число строк, самая длинная строка поддерева и ещё несколько полей.
- Сводки детей складывает операция `add_summary`, у неё есть нейтральный элемент: это моноид из части 2.

Т. Ball и др., «Rope & SumTree», Zed Blog, 2024; [zed-industries/zed](#), [crates/sum_tree/src/sum_tree.rs](#), [crates/rope/src/rope.rs](#).

Самая длинная строка: максимуму нужно дерево

В сводке Zed есть самая длинная строка поддерева.

- Длину и число переводов можно считать префиксными суммами: сумма на куске = разность двух префиксов. Нужно вычитание.
- У максимума обратной операции нет: по максимуму всего текста и максимуму его начала максимум конца не узнать.
- Дерево обходится без вычитания: сводки детей только складываются. Годится любой моноид, и максимум тоже.

`zed-industries/zed`, `crates/rope/src/rope.rs`: `TextSummary` и её сложение (`add_assign`).

Что осталось медленным: работа внутри листа

Спуск по дереву доводит до нужного листа за $O(\log n)$. Дальше внутри листа (до 128 байт) надо найти строку и колонку для offset.

До осени 2024 года Zed делал это циклом: идти по байтам листа до offset и считать переводы строк.

Этот перевод — горячий путь: по словам авторов, он выполняется «сотни, если не тысячи раз в секунду», пока вы ходите по файлу и правите его. Цикл по 128 байтам на каждый вызов — заметная цена.

T. Ball, A. Scandurra, «Rope Optimizations, Part 1», Zed Blog, 18.11.2024, раздел «The Problem».

Было: цикл по байтам листа

```
251 static Point offset_to_point_loop(const char* t, size_t n, size_t off) {
252     Point p{0, 0};
253     for (size_t i = 0; i < n && i < off; ++i) {
254         if (t[i] == '\n') { p.column = 0; ++p.row; } else { ++p.column; }
255     }
256     return p;
257 }
258
```

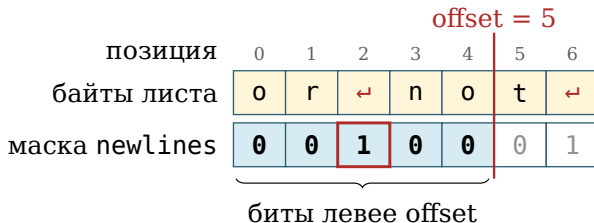
Идём от начала листа до offset: перевод строки — новая строка, колонка с нуля; любой другой байт — колонка +1. До 128 шагов и ветвление на каждом.

Это наша версия их цикла на C++ — чтобы измерить на своей машине. У Zed тот же цикл написан на Rust.

Наш код, стенд — AMD Zen 5, одно ядро, gcc -O2.

Стало: маска переводов строк

При создании листа строится 128-битное число `newlines`: бит i равен 1, если байт i — перевод строки.



- **строка** = число единиц левее offset (здесь 1); их считает одна инструкция `popcount`;
- **колонка** = `offset` – (позиция последней такой единицы + 1) = $5 - 3 = 2$; позицию даёт инструкция `clz` — число нулей в старших битах.

Маска в коде

```
266 static Point offset_to_point_mask(u128 newlines, size_t off) {
267     u128 mask = (off >= MAXB) ? ~(u128)0 : (((u128)1 << off) - 1);
268     u128 m = newlines & mask;
269     uint32_t row = (uint32_t)(__builtin_popcountll((u64)m +
270                                     __builtin_popcountll((u64)(m >> 64))));
271     u64 nl_ix = 128 - clz128(m);
272     uint32_t column = (uint32_t)(off - nl_ix);
273     return Point{row, column};
274 }
275
```

- m — только биты левее $offset$; row — их число;
- $nl_ix = 128 - clz(m)$ — начало текущей строки;
- $column = off - nl_ix$.

Наш код; у Zed — `crates/rope/src/chunk.rs, Chunk::offset_to_point`.

Что ускорили: та же идея уровнем ниже

где	без сводки	со сводкой
дерево	пройти весь текст до offset: $O(n)$	спуск по длинам и числам переводов в узлах: $O(\log n)$
лист	цикл по байтам до offset: до 128 шагов	маска, построенная один раз; popcount и clz

Новой структуры данных здесь нет. И в узлах дерева, и в листе приём один: **заранее посчитанная сводка вместо прохода.**

Замер: насколько маска быстрее цикла

наш замер: 17.88 нс $\longrightarrow$ **6.00 нс, в 3.0 раза**

Один вызов перевода внутри листа в 128 байт; каждый следующий offset вычисляется из предыдущего ответа; лучший из 1400 отдельных замеров.

У Zed в посте: 57 нс $\rightarrow$ 1.05 нс, в 57 раз по данным авторов, на их железе. Программы, которой они это мерили, в их репозитории нет, поэтому повторить их замер точно нельзя. Во всём редакторе, по их же словам, эффект скромнее: «до 70% быстрее».

Наш стенд: AMD Zen 5, одно ядро, gcc -O2. Их числа: «Rope Optimizations, Part 1», Zed Blog, 18.11.2024.

Как мы знаем, что стало быстрее

«Быстрее в N раз» — это вывод из замера. Каким должен быть замер, чтобы вывод был верным?

Замер из PR Zed: таблица, а не одно число

В посте — «до 70% быстрее». Это одна строка таблицы из PR, где изменение внесли. Каждая строка — отдельная операция над Rope на тексте в 4096 байт; «время» — насколько оно изменилось.

операция	что делает	время	вывод
point_to_offset	(строка, колонка) → смещение	−71.79%	быстрее
slice	вырезать кусок текста	−48.53%	быстрее
clip_point	сдвинуть позицию на настоящий символ	−45.87%	быстрее
append	приклеить один Rope к другому	−94.81%	быстрее
push	дописать текст в конец	−3.10%	быстрее
bytes_in_range	пройти байты куска текста	+0.60%	в пределах шума
chars	пройти все символы текста	+9.40%	медленнее

Одна операция стала медленнее — и автор PR написал об этом сам.

по данным авторов, на их железе

PR [zed-industries/zed#19913](#), «Speed up point translation in the Rope», 2024.

Вывод: замер — это таблица с условиями

Число «в 57 раз» или «в 3 раза» без условий нельзя ни проверить, ни повторить. Рядом с ним должно быть:

1. **что мерили:** какая операция, на каких данных, какого размера;
2. **на чём:** процессор, компилятор и его флаги;
3. **как:** сколько отдельных замеров и что из них взято — лучший, медиана;
4. **все строки:** и «быстрее», и «без изменений», и «медленнее», как в таблице PR.

Итоги

Куда этот приём ушёл дальше и одна таблица дня.

Обучение с подкреплением: что хранит агент

- **Обучение с подкреплением:** программа-агент (например, играет в игру) пробует действия, получает награду и учится выбирать действия, за которые награда больше.
- Агент хранит прошлый опыт — миллионы записей «что видел, что сделал, что получил» — и учится на записях, выбранных случайно.
- **Приоритетный выбор:** чаще брать записи, на которых агент ошибся сильнее. У каждой записи — вес.

Где здесь дерево отрезков

- **OpenAI Baselines** — открытая библиотека OpenAI с алгоритмами обучения с подкреплением. Веса записей там лежат в дереве отрезков с суммой. Выбрать запись с вероятностью, пропорциональной весу, — взять случайное число от 0 до общей суммы и спуститься по суммам: $O(\log n)$. Вес изменился — правка, $O(\log n)$.
- Класс дерева там принимает операцию и нейтральный элемент — моноид; второе такое дерево хранит минимум весов.

T. Schaul и др., «Prioritized Experience Replay», ICLR 2016; github.com/openai/baselines, `baselines/common/segment_tree.py`.

Сводка в узле — не только в редакторах

- **Компьютерная графика.** BVH — дерево ограничивающих объёмов: в узле — коробка, охватывающая всё поддереву. Луч проверяют на пересечение сначала с коробкой, а не с каждым треугольником.
- **Редакторы.** Кроме Zed, Rope хранит текст, например, в редакторе Helix.

J. Bikker, «How to build a BVH», 2022; T. Ball и др., «Rope & SumTree», 2024.

Итог дня: структура — что в узле — что умеет

структура	что в узле	что умеет
AVL	высота поддерев	поиск, вставка, удаление: $O(\log n)$
2-3-4	от 1 до 3 ключей	то же; все листья на одной глубине
красно-чёрное	цвет узла	то же; это 2-3-4 в двоичной записи
дерево отрезков	агрегат отрезка	агрегат на отрезке и правка: $O(\log n)$
Фенвик	сумма отрезка длины $\text{lowbit}(i)$, кончая i	префиксная сумма и правка: $O(\log n)$
декартово (treap)	ключ и приоритет	split и merge: $O(\log n)$ в среднем
неявное декартово	размер поддерев	вставка по позиции: $O(\log n)$ в среднем
Rope	длина, число переводов	смещение $\rightarrow$ строка, вставка: $O(\log n)$

Во всех строках одно и то же: в узле — сводка по поддереву, и операция либо ищет по сводкам, либо чинит их после правки.

Что сегодня не вошло

- **Ленивое распространение** — правка целого отрезка (например, «прибавить 5 ко всем элементам с 10-го по 90-й») за $O(\log n)$ в дереве отрезков.
- **Персистентные деревья** — доступ ко всем прежним версиям структуры; на этом стоят снимки текста в Zed.
- **Splay-деревья** — ещё один способ балансировки, без хранимых высот и цветов.
- **Дерево отрезков и Фенвик в двух измерениях** — суммы по прямоугольникам.
- **Арифметическое кодирование**, ради которого придуманы Фенвик и Рябко, — в следующем семестре.